



The benefits of Indirection

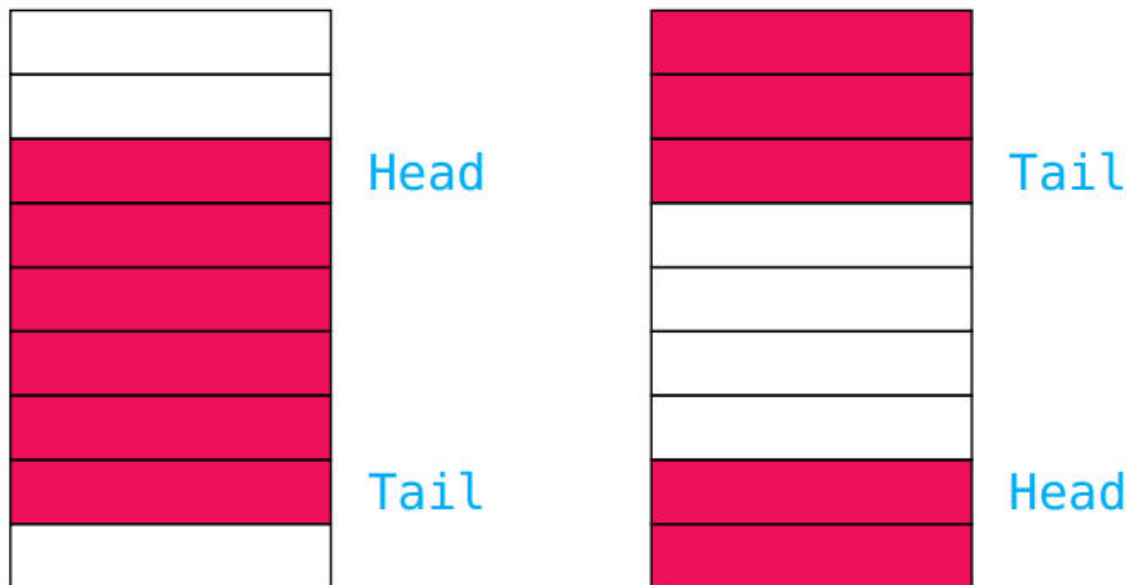
▼ Type	 Lecture
📅 Date	@February 4, 2022
☰ Lecture #	1
🔗 Lecture URL	https://youtu.be/LY7TRCI3z24
🔗 Notion URL	https://21f1003586.notion.site/The-benefits-of-Indirection-866f9be0ef8046bb922186b620d02b8d
# Week #	6

Abstract data types

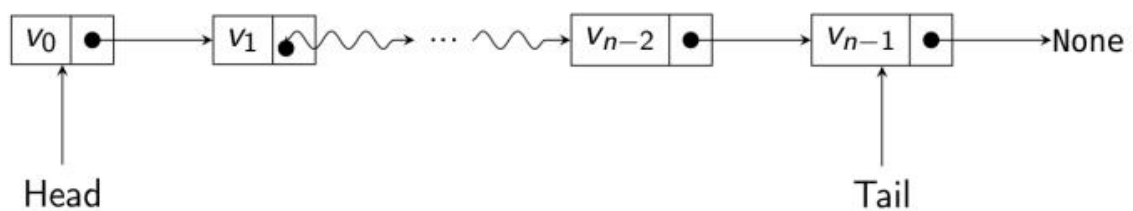
- It separates the public interface from the private implementation
- For example, a generic Queue

```
public class Queue<E> {  
    public void add(E element) { ... }  
    public E remove() { ... }  
    public int size() { ... }  
    ...  
}
```

- Concrete implementation could be a circular array



- or a Linked List



- The person who implemented the class `Queue` can choose either one
- The public interface remains un-changed

-
- But the `interface` does not capture other aspects, like ...
 - Efficiency
 - Circular array is better — one time storage allocation
 - Flexibility
 - Linked List is better — circular array has a bounded size

Multiple implementations

- Two separate implementations

```
public class CircularArrayQueue<E> {
    public void add(E element) { ... }
    public E remove() { ... }
    public int size() { ... }
    ...
}
```

```
public class LinkedListQueue<E> {
    public void add(E element) { ... }
    public E remove() { ... }
    public int size() { ... }
    ...
}
```

- The user gets to decide ...

```
CircularArrayQueue<Date> dateq;
LinkedListQueue<String> stringq;

dateq = new CircularArrayQueue<Date>();
stringq = new LinkedListQueue<String>();
```

- But we'd have to change the declaration of `dateq` if we later felt the need for a flexible size `dateq`
 - Also, we'd have to make changes to every function header, auxiliary variables, ... etc. associated with it

Adding indirection

- Instead of whatever we did earlier, create a `Queue` interface
 - The concrete implementations then implement the interface

```
public interface Queue<E> {
    abstract void add(E element);
    abstract E remove();
    abstract int size();
}

public class CircularArrayQueue<E> implements Queue<E> {
    public void add(E element) { ... }
    public E remove() { ... }
    public int size() { ... }
    ...
}
```

```
public class CircularArrayQueue<E> implements Queue<E> {
    public void add(E element) { ... }
    public E remove() { ... }
    public int size() { ... }
    ...
}
```

- Use the `interface` to declare variables

```
Queue<Date> dateq;
Queue<String> stringq;

dateq = new CircularArrayQueue<Date>();
stringq = new LinkedListQueue<String>();
```

- The benefits of **indirection**
 - To use a different implementation for `dateq`
 - Only need to update the instantiation

Summary

- Use interfaces to flexibly choose between multiple concrete implementations
- Interfaces add a level of indirection
- Indirection in real life ...
 - Organization provides senior staff with an office car
 - Concrete → each official has an assigned car — what if it breaks down?
 - Indirection → a pool of office cars, use any that is available
 - Don't want to maintain a pool of cars? Contract with a taxi service
 - Don't want to negotiate tenders? Re-imburse the taxi bills

“Fundamental theorem of software engineering”

All problems in Computer Science can be solved by another level of indirection

- *Butler Lampson, Turing Award 1992*



Collection

▼ Type	 Lecture
📅 Date	@February 4, 2022
☰ Lecture #	2
🔗 Lecture URL	https://youtu.be/-EwtNZTsOVw
🔗 Notion URL	https://21f1003586.notion.site/Collection-e4cdb6c129cc46c6ad9bc49269b06c72
# Week #	6

Built-in data types

- Most programming languages provide built-in collective data types
 - Arrays
 - Lists
 - Dictionaries
 - ...
- Java originally had many such pre-defined classes

- `Vector`
- `Stack`
- `Hashtable`
- `Bitset`
- `...`
- We'd often choose the one we need, but changing a choice usually required multiple updates
- Instead, we'd organize these data structures by functionality
- Create a hierarchy of abstract interfaces and concrete implementations
 - Provide a level of *indirection*

The `Collection` interface

- The `Collection` interface abstracts properties of grouped data
 - `Arrays`
 - `Lists`
 - `Sets`
 - `...`
 - But not key-value structures like dictionaries
- `add()` — add to the collection
- `iterator()` — get an object that implements `Iterator` interface
- Use the iterator to loop through the elements

```
public interface Collection<E> {
    boolean add(E element);
    Iterator<E> iterator();
    ...
}

public interface Iterator<E> {
    E next();
    boolean hasNext();
    void remove();
    ...
}

Collection<String> cstr = new ...;
```

```

Iterator<String> iter = cstr.iterator();
while (iter.hasNext()) {
    String element = iter.next();
    // do something with the element
}

```

Using iterators

- Use iterator to loop through the elements

```

Collection<String> cstr = new ...;
Iterator<String> iter = cstr.iterator();
while (iter.hasNext()) {
    String element = iter.next();
    // do something with the element
}

```

- Java later added the “for each” loop
 - It implicitly creates an iterator and runs through it

```

Collection<String> cstr = new ...;
for(String element : cstr) {
    // do something with element
}

```

- Generic functions to operate on collections

```

public static <E> boolean contains(Collection<E> c, Object obj) {
    for(E element : c) {
        if(element.equals(obj)) {
            return true;
        }
    }
    return false;
}

```

- *Explained later: How does the following line work*

- `if(element.equals(obj))`

Removing elements

- Iterator also has a `remove()` method

```
public interface Iterator<E> {
    E next();
    boolean hasNext();
    void remove();
    ...
}
```

- It removes the element that was last accessed by `next()`
 - When the iterator starts, the next points to the first element
 - This means, the iterator is standing over, basically, nothing
 - When we call the `next()` for the first time ...
 - The iterator passes the first element and stands right before the second element pointing to it
 - Now, if we call the `remove()` method, it will delete the first element as it was the last accessed element by `next()`
 - Now, we CANNOT call `remove()` again immediately after the previous deletion, because the iterator has not passed any new element and is still pointing to the 2nd element
 - This will cause an error

```
Collection<String> cstr = new ...;
Iterator<String> iter = cstr.iterator();
while(iter.hasNext()) {
    String element = iter.next();
    // Delete the element if it has some property
    if(property(element)) {
        element.remove();
    }
}
```

- To remove consecutive elements, we must interleave a `next()`

```
// The following code will cause an error, described above
iter.remove();
iter.remove();
```

```
// The following code is fine
iter.remove();
```



```
iter.next();
iter.remove();
```

- To remove the first element, we need to access it first

```
// To remove the first element in cstr
iter.next();
iter.remove();
```

The `Collection` interface — contd.

- How does the following line work?

- `if(element.equals(obj))`

```
public static <E> boolean contains(Collection<E> c, Object obj) {
    for(E element : c) {
        if(element.equals(obj)) {
            return true;
        }
    }
    return false;
}
```

- Actually, `Collection` defines a much larger set of abstract methods

```
public interface Collection<E> {
    boolean add(E element);
    Iterator<E> iterator();
    int size() boolean isEmpty();
    boolean contains(Object obj);
    boolean containsAll(Collection<?> c);
    boolean equals(Object other);
    boolean addAll(Collection<? extends E> from);
    boolean remove(Object obj);
    boolean removeAll(Collection<?> c);
}
```

- `addAll(from)` adds elements from a compatible collection
- `removeAll(c)` removes elements present in `c`
- This `remove()` is quite different from the one present in `Iterator`
- So, in order to implement the `Collection` interface, we need to implement all these methods

The `AbstractCollection` class

- To implement `Collection`, we need to implement all the methods required (mentioned above)
- So a correct solution — provide default implementations in the interface itself
- So, `AbstractCollection` is a abstract class which implements `Collection`

```
public abstract class AbstractCollection<E> implements Collection<E> {  
    ...  
    public abstract Iterator<E> iterator();  
    public boolean contains(Object obj) {  
        for(E element : this) {  
            if(element.equals(obj)) {  
                return true;  
            }  
        }  
        return false;  
    }  
    ...  
}
```

- So, the concrete class now extends the `AbstractCollection`
 - We need to define `iterator()` based on the internal representation
 - We can choose to override `contains()`

Summary

- The `Collection` interface captures abstract properties of collections
 - Add an element, create an iterator, ...
- We can use for each loop to avoid explicit iterator
- Write generic functions that operate on collections
- `Collection` defines many additional abstract functions, tedious if we have to implement each one of them
- `AbstractCollection` provides default implementations to many functions required by `Collection`
- Concrete implementations of collections extend `AbstractCollection`



Concrete Collections

▼ Type	 Lecture
📅 Date	@February 5, 2022
☰ Lecture #	3
🔗 Lecture URL	https://youtu.be/XPzVy2DyO6E
🔗 Notion URL	https://21f1003586.notion.site/Concrete-Collections-949ea83681704e34a60baed60ec6dc1f
# Week #	6

Built-in data types

- The `Collection` interface abstracts properties of grouped data
 - Arrays, lists, sets, ...
 - But not the key-value data structures like dictionaries
- Collections can be further organized based on additional properties like ...
 - Are the elements ordered?
 - Are duplicates allowed

- Are there constraints on how elements are added, removed?
- These are captured by interfaces that extend `Collection`
 - Interface `List` for ordered collections
 - Interface `Set` for collection without duplicates
 - Interface `Queue` for ordered collections with constraints on addition and deletion

The `List` Interface

- An ordered collection can be accessed in two ways
 - Through an iterator
 - By position — random access
- Additional functions for random access
- `ListIterator` extends `Iterator`
 - `void add(E element)` to insert an element before the current index
 - `void previous()` to go to the previous element
 - `boolean hasPrevious()` checks whether it is legal to go backwards

```
public interface List<E> extends Collection<E> {
    void add(int index, E element);
    void remove(int index);
    E get(int index);
    E set(int index, E element);

    ListIterator<E> listIterator();
}
```

The `List` interface and random access

- Random access is not equally efficient for all the ordered collections
 - In an array, we can compute the location of element at index `i`
 - In a linked list, we must start at the beginning and traverse `i` links
- Tagging interface `RandomAccess`
 - It tells us whether a `List` supports random access or not
 - We can choose algorithmic strategy based on this

```

public interface List<E> extends Collection<E> {
    void add(int index, E element);
    void remove(int index);
    E get(int index);
    E set(int index, E element);

    ListIterator<E> listIterator();
}

if(c instanceof RandomAccess) {
    // use random access algorithm
} else {
    // use sequential access algorithm
}

```

The **AbstractList** interface

- **AbstractCollection** is a usable version of **Collection**
- Correspondingly, **AbstractList** extends **AbstractCollection**
 - Inherits default implementations
- **AbstractSequentialList** extends **AbstractList**
 - A further sub-class to distinguish lists without random access
- Concrete generic class **LinkedList<E>** extends **AbstractSequentialList**
 - Internally, the usual flexible linked list
 - It is efficient to add and remove elements at arbitrary positions
- Concrete generic class **ArrayList<E>** extends **AbstractList**
 - It has flexible size array
 - It supports random access

Using concrete list classes

- Concrete generic class **LinkedList<E>** extends **AbstractSequentialList**
 - No random access
 - But random access methods of **AbstractList** are still available
 - The following loop with execute a fresh scan from the start to element **i** in each iteration

```
LinkedList<String> list = new ...;
for(int i = 0; i < list.size(); ++i) {
    // do something with list.get(i)
}
```

- Two versions of `add()` are available
 - `add()` from `Collection` appends to the end of the list
 - `add()` from `ListIterator` inserts a value before the current position of the iterator
- In `Collection`, `add()` returns boolean — which represents “did the `add` update the collection?”
 - `add()` may not update a set, always work for lists
- `add()` is `ListIterator` returns `void`

The `Set` interface

- A set is a collection without duplicates
- `Set` interface is identical to `Collection`, but behaviour is more constrained
 - `add()` should have no effect, and return `false` if the element already exists
 - `equals()` should return `true` if contents match after disregarding order
- Use `Set` to constraint values to satisfy additional constraints
- `Set` implementations are typically designed to allow efficient membership tests
- Ordered collections loop through a sequence to find an element
- Instead, map the value to its position
 - Hash function
- Or arrange values in a 2D structure
 - Balanced search tree
- As usual, concrete set implementations extend `AbstractSet`, which extends `AbstractCollection`

Concrete sets

- `HashSet` implements a hash table

- Underlying storage is an array
- Map value `v` to a position `h(v)`
- If `h(v)` is unoccupied, store `v` at that position
- Otherwise, collision — different strategies to handle this case
- Checking membership is fast — check if `v` is at position `h(v)`
- It is unordered, but supports `iterator()`
 - Scan the elements in unspecified order
 - Visit each element exactly once
- `TreeSet` uses a tree representation
 - Values are ordered
 - Maintains a sorted collection
- Iterator will visit elements in sorted order
- Insertion is more complex than a hash table
 - Time $O(\log n)$ if the set has `n` elements

The `Queue` interface

- Ordered, remove front, insert rear
- `Queue` interface supports the following
 - `boolean add(E element);`
 - `E remove();`
 - If the queue is full, `add()` flags an error
 - If queue empty, `remove()` flags an error
- Gentler versions of `add()`, `remove()`
 - `boolean offer(E element);`
 - `E poll();`
 - Return `false` or `null`, respectively, if not possible
- Inspect the head, no update
 - `E element(); // Throws exception`

- `E peek(); // Returns null`
- Interface `Deque`, double ended queue
 - `boolean addFirst(E element);`
 - `boolean addLast(E element);`
 - `boolean offerFirst(E element);`
 - `boolean offerLast(E element);`
 - `E pollFirst();`
 - `E pollLast();`
 - `E getFirst();`
 - `E getLast();`
 - `E peekFirst();`
 - `E peekLast();`
- Interface `PriorityQueue`
 - `remove()` returns highest priority item
- Concrete implementations
 - `LinkedList` — implements `Queue`
 - `ArrayDeque` — circular array `Deque`

Summary

- Different types of `Collection` are specified by sub-interfaces
 - `List`, `Set`, `Queue`
- `List` allows random access, more functional `ListIterator`
- `Set` constraints collection to not have duplicates
- `Queue` supports restricted add and remove methods
- Each interface has corresponding version under `AbstractCollection`
- Concrete implementations extend `AbstractList`, `AbstractSet` and `AbstractQueue`



Maps

▼ Type	 Lecture
📅 Date	@February 5, 2022
☰ Lecture #	4
🔗 Lecture URL	https://youtu.be/fWXf5aI3Cr0
🔗 Notion URL	https://21f1003586.notion.site/Maps-6569131529294cf48862b4d54d044f0c
# Week #	6

Maps

- The `Collection` interface abstracts properties of grouped data
 - Arrays, lists, sets, ...
 - But not key-value structures like dictionaries

```
public interface Map<K,V> {  
    V get(Object key);  
    V put(K key, V value);  
  
    boolean containsKey(Object key);  
    boolean containsValue(Object value);  
}
```

```
    ...  
}
```

- Key-value structures come under the `Map` interface
 - Two type parameters
 - `K` is the type for keys
 - `V` is the type for values
 - `get(k)` fetches the value for the key `k`
 - `put(k, v)` updates the value for the key `k`
- The keys form a set
 - Only one entry per key-set
 - Assigning a fresh value to existing key will overwrite the old value
 - `put(k, v)` returns the previous value associated with `k`, or `null`

Updating a map

- Key-value stores are useful to accumulate quantities
 - Frequencies of words in a text
 - Total runs in a tournament
- The initialization problem
 - Update the value if the key exists
 - Otherwise, create a new entry
- Map has the following default method

```
V getOrDefault(Object key, V defaultValue)
```
- For instance

```
Map<String, Integer> scores = ...;  
int score = scores.getOrDefault(bat, 0);
```

sets `score` to `0` if the key `bat` is not present
- Now, we can update the entry for the key `bat` as follows

```
scores.put(bat, scores.getOrDefault(bat, 0) + newscore);
```
- Alternatively, use `putIfAbsent()` to initialize a missing key

```
scores.putIfAbsent(bat, 0);
```

```
scores.put(bat, scores.get(bat) + newscore);
```

- Or use `merge()`

```
scores.merge(bat, newscore, Integer::sum);
```

- Initialize to `newscore` if there is no such key `bat`
- Otherwise, combine the current value with `newscore` using `Integer::sum`

Extracting keys and values

- Methods to extract the keys and values

```
Set<K> keySet();  
Collection<V> values();  
Set<Map.Entry<K,V>> entrySet();
```

- Keys form a `Set` while values from an arbitrary `Collection`
- Key-value pairs form a set over a special type `Map.Entry`
- Java calls these views
- We can now iterate over a `Map`

```
Set<String> keys = strmap.keySet();  
for(String key : keys) {  
    // do something with the key  
}
```

- Use `entrySet()` to operate on the key and associated value without looking up map again

```
for(Map.Entry<String, Employee> entry : staff.entrySet()) {  
    String k = entry.getKey();  
    Employee v = entry.getValue();  
    // do something with k, v  
}
```

Concrete implementation of Map

`HashMap`

- It works similar to `HashSet`
- It uses a hash table to store the keys and values

- There is no fixed order over which keys returned by `keySet()`

`TreeMap`

- It is similar to `TreeSet`
- It uses a balanced search tree to store the keys and values
- An iterator over `keySet()` will process the keys in a sorted order

`LinkedHashMap`

- It remembers the order in which keys were inserted
- Hash table entries are also connected as a (doubly) linked list
- The iterators over both `keySet()` and `value()` enumerate in the order of insertion
- We can also use access order
 - Each `get()` and `put()` moves key-value pair to the end of the list
 - Process the entries in least recently used order — scheduling, caching
- There is a similar `LinkedHashSet`

Summary

- The `Map` interface captures properties of key-value stores
 - `get(), put(), containsKey(), containsValue(), ...`
- Parameterized by two types variables, `K` for keys and `V` for values
- Keys form a set
- Different ways to update a key entry, depending on whether the key already exists
 - `getOrDefault(), putIfAbsent(), merge()`
- Extract keys as a `Set`, values as a `Collection`, key-value pairs as a `Set`
 - `keySet(), values(), entrySet()`
- We use these “views” to iterate over all the key-value pairs in the map
- Concrete implementations → `HashMap, TreeMap, LinkedHashMap`